ARC
Artefact Replication & Calibration Engine

Project Results & Progress Report

Anti-Sandbox Fingerprinting: Realistic Windows 11 VM Personalisation for Dynamic Malware Analysis

Team Member	Domain
Sumukha	Core Infrastructure + Filesystem
Snehal	Registry + Event Logs + Anti-Fingerprint
Raghottam	Browser + Applications + Evaluation

March 2026

Revision 1.0

0 Contents

1 Introduction **2**

2 System Architecture **2**
2.1 High-Level Design . . . 2
2.2 Layer Descriptions . . . 3

3 Artefact Generation Pipeline **3**
3.1 Pipeline Overview . . . 4
3.2 Types of Artefact Generation . . . 5
3.2.1 Registry Artefacts . . . 5
3.2.2 Event Log Artefacts . . . 5
3.2.3 Hardware & Anti-Fingerprint Artefacts . . . 5
3.2.4 Browser Artefacts . . . 6
3.2.5 Filesystem Artefacts . . . 6

4 Profile Coverage & Evaluation Results **6**
4.1 Sandbox Signal Categories Addressed . . . 6
4.2 Per-Profile Artefact Density . . . 7
4.3 Component Implementation Status . . . 8
4.4 Test Coverage Summary . . . 8

5 Timeline & Milestone Evaluation **9**
5.1 Planned Timeline vs. Actual Progress . . . 9
5.2 Milestone Assessment . . . 10

6 Known Limitations **11**

7 Remaining Work & Priorities **11**

8 Conclusion **12**

A Profile YAML Schema Summary **13**

B Consistency Check Pass Criteria **13**

1 Introduction

Modern dynamic malware analysis platforms execute suspicious binaries inside virtual machines (VMs) and observe their runtime behaviour. A significant fraction of contemporary malware employs *sandbox-evasion* techniques: before executing its primary payload the sample inspects the host environment for telltale signs of virtualisation and analysis, and sleeps or exits harmlessly if any are detected.

Common detection vectors include:

- **Registry artefacts** — presence of VirtualBox/VMware driver service keys; missing user-activity keys (UserAssist, RecentDocs).
- **Event log sparseness** — empty or stub `.evtx` files whose timestamps all cluster on the same day.
- **Hardware identifier strings** — BIOS vendor "SeaBIOS", disk model "VBOX HARDDISK", GPU "Standard VGA".
- **Filesystem poverty** — no browser history, no documents, no downloaded files, bare user-profile directory.

ARC (Artefact Replication & Calibration) addresses each vector systematically. Given a *mounted* Windows 11 image and a *profile YAML* that encodes a simulated user persona, ARC writes realistic, internally consistent artefacts — registry keys, event-log records, browser history, downloaded files, application traces — that collectively defeat static VM-detection heuristics.

This report documents the results achieved to date, the evaluation methodology, the artefact generation pipeline, and the work remaining to reach a fully functional prototype.

2 System Architecture

2.1 High-Level Design

Figure 1 shows the end-to-end data-flow from a profile YAML file to a personalised Windows 11 image.

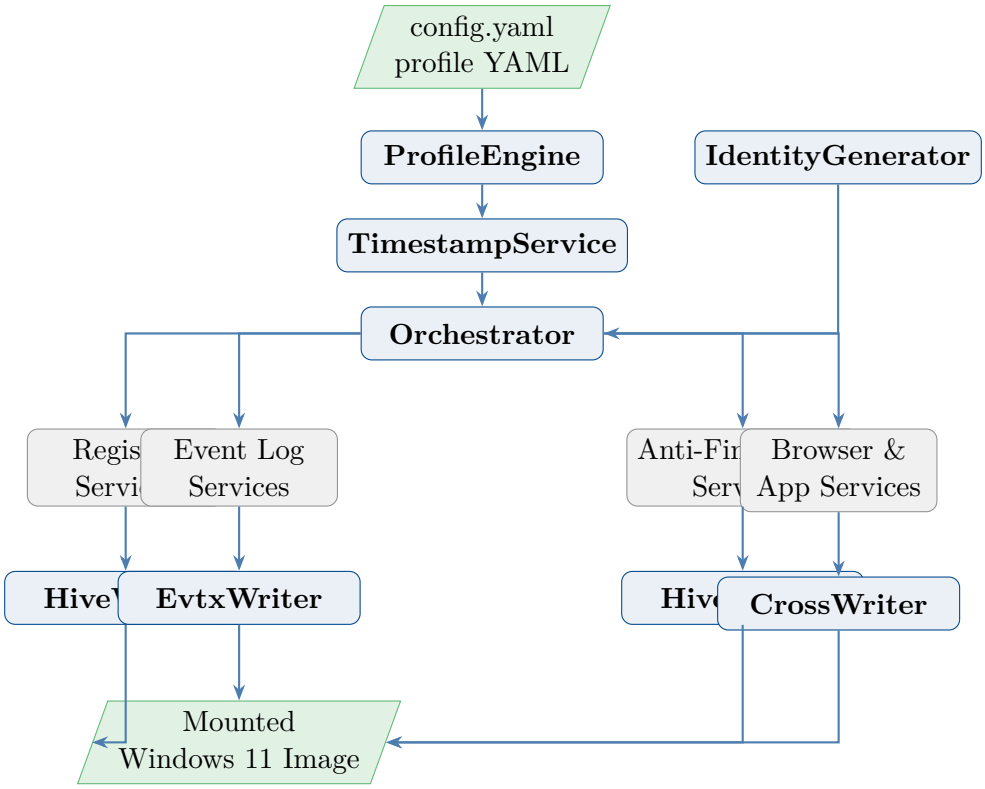

Figure 1: ARC end-to-end data-flow. The Orchestrator coordinates four service families; each family writes to the mounted image through a typed I/O adapter.

2.2 Layer Descriptions

Core Layer. `ProfileEngine` deep-merges the base YAML with the chosen persona YAML and validates the result through a frozen Pydantic `ProfileContext` model, catching schema violations at load time. `IdentityGenerator` produces a deterministic `IdentityBundle` (computer name, username, hardware serial numbers) seeded on `(computer_name, profile_type)` so that repeated runs are reproducible. `TimestampService` yields chronologically sorted datetime lists spread over a configurable look-back window (60, 120, or 180 days).

Services Layer. All service classes inherit from `BaseService` (an ABC) and receive their dependencies — `HiveWriter` or `EvtxWriter` plus `AuditLogger` — via constructor injection. No service accesses the filesystem directly; every write flows through a typed adapter, enabling full mock-based unit testing.

Binary I/O Adapters. `HiveWriter` reads the existing hive cell map with *regipy* and patches values in-place at the binary level. `EvtxWriter` constructs binary-valid EVTX files from scratch — 4096-byte file header (`ElfFile\0`), 65536-byte chunks (`ElfChnk\0`), variable-length records — with CRC32 checksums per the Microsoft EVTX specification.

3 Artefact Generation Pipeline

3.1 Pipeline Overview

Figure 2 details the sequence in which the Orchestrator invokes services, showing inter-service dependencies.

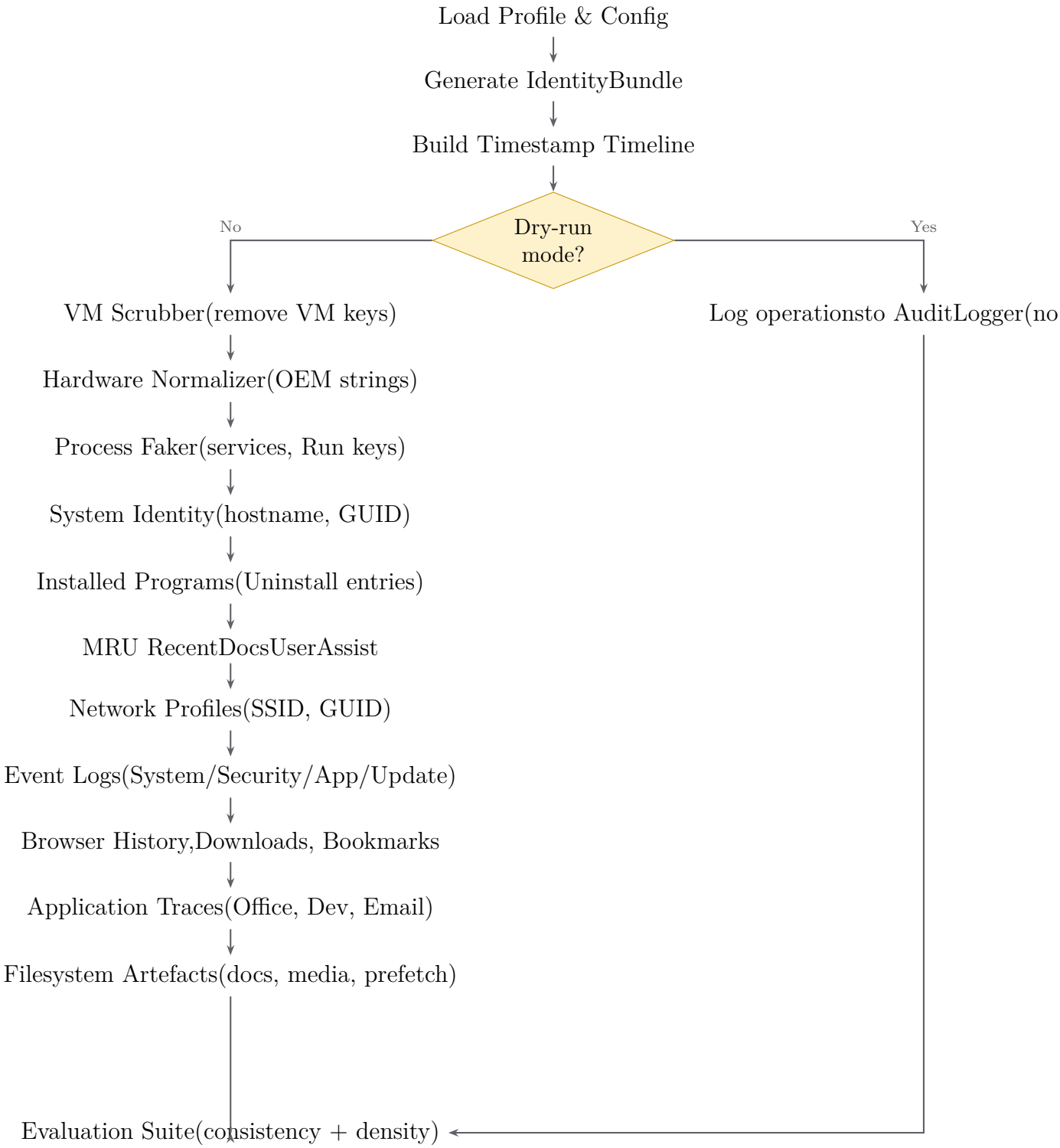

Figure 2: Artefact generation pipeline. Services execute in dependency order: VM scrubbing must precede hardware normalisation; registry writes precede event-log writes; filesystem writes are last.

3.2 Types of Artefact Generation

ARC generates artefacts across five distinct subsystems, each targeting a specific class of sandbox detection heuristic.

3.2.1 Registry Artefacts

Registry-based checks are the most prevalent sandbox signal. ARC populates five key areas of the Windows registry hive files (`NTUSER.DAT`, `SOFTWARE`, `SYSTEM`):

- **SystemIdentity** — `ComputerName`, `ProductId`, machine GUID, BIOS serial, disk serial number.
- **InstalledPrograms** — 8–16 `Uninstall` entries with deterministic GUIDs derived from an SHA-256 seed, covering profile- appropriate applications (Office, Chrome, Docker, Git, VS Code, etc.).
- **MruRecentDocs** — 15-entry MRU lists per file extension across `RecentDocs`, `OpenSaveMRU`, and `LastVisitedMRU`.
- **UserAssist** — ROT-13-encoded execution-trace records for each installed application, with realistic run-count and focus-time data.
- **NetworkProfiles** — a plausible Wi-Fi/LAN profile keyed under a unique GUID with SSID, first/last connect timestamps.

3.2.2 Event Log Artefacts

Windows event logs (`.evtx`) are inspected by analysis platforms to assess machine age and user activity. ARC generates binary-valid EVTX files from scratch:

- **SystemLog** — boot/shutdown cycles (EID 6005, 6006), service start/stop events (EID 7036), Windows time-sync (EID 7001). Approx. 28–40 records depending on profile.
- **SecurityLog** — logon (EID 4624), special-privilege logon (EID 4672), Kerberos ticket requests (EID 4769, office/developer profiles only), audit-policy change (EID 4907). 3–6 sessions.
- **ApplicationLog** — MSI install events (EID 11707) for each installed application, paired application crash/recovery events (EID 1000/1001).
- **UpdateArtifacts** — 8–18 KB Windows Update entries pairing registry timestamps with EVTX Install events (EID 19/20/43/44).

All event timestamps are drawn from `TimestampService`'s sorted chronological list, guaranteeing monotonically increasing records within each log file.

3.2.3 Hardware & Anti-Fingerprint Artefacts

Three services remove VM-indicator strings and replace them with realistic OEM data from `data/hardware_models.json`:

- **VmScrubber** — deletes VirtualBox and VMware driver service keys from the SYSTEM hive; patches VM vendor strings ("VBOX", "VMWARE", "QEMU", "innotek") in any hive value.
- **HardwareNormalizer** — overwrites BIOS manufacturer, motherboard product name, disk model, and GPU adapter string with values drawn at random from the OEM hardware catalogue.

- **ProcessFaker** — populates 37 real Windows service entries in the SYSTEM hive; writes profile-specific `Run` keys (e.g. Spotify for `home_user`, Docker + Slack for `developer`).

3.2.4 Browser Artefacts

Browser history and download records are among the strongest indicators of genuine user activity:

- **BrowserProfile** — creates the Chrome/Edge profile directory scaffold and writes `Preferences` JSON (locale, first-run time, sync GUID).
- **History** — populates the Chrome `History` SQLite database (`urls`, `visits`, `keyword_search_terms` with visit counts and timestamps drawn from the configured URL category buckets and `daily_avg_sites`.
- **Downloads** — inserts rows into `downloads` and `downloads_url_chains`; places zero-byte placeholder files in the `Downloads/` folder.

3.2.5 Filesystem Artefacts

File-system presence (user documents, media, prefetch files) defeats heuristics that check for an empty `C:\Users\<name>` directory:

- **CrossWriter** — recursive, schema-validated directory and file writer with Win32 attribute and timestamp injection.
- **UserDirectory** — scaffolds the standard Windows user-profile directory tree.
- **DocumentGenerator** — generates realistic `.docx`, `.xlsx`, `.pdf`, and `.txt` files with embedded author/company metadata.
- **MediaStub** — produces JPEG/PNG files with plausible EXIF tags (camera model, GPS coordinates, capture date).
- **Prefetch** — writes synthetic `.pf` Prefetch files to `C:\Windows\Prefetch`.
- **RecentItems** — creates `.lnk` shortcut files and Jump-List `.automaticDestinations` entries.
- **RecycleBin** — produces paired `$I` / `$R` artefacts in `$Recycle.Bin\{user-SID}`.

4 Profile Coverage & Evaluation Results

4.1 Sandbox Signal Categories Addressed

Table 1 maps each known detection vector to the ARC service that mitigates it.

Table 1: Sandbox signal categories and ARC mitigations.

Signal Category	Detection behaviour	ARC Service
VM driver service keys	`VBoxGuest` etc. present in SYSTEM hive	`VmScrubber`
VM vendor strings	String scan for "VBOX", "VMWARE"	`VmScrubber`
Generic hardware IDs	BIOS vendor = "SeaBIOS" or "innotek"	`HardwareNormalizer`
Missing installed programs	`Uninstall` key empty or minimal	`InstalledPrograms`
No recent documents	`RecentDocs` MRU empty	`MruRecentDocs`
No UserAssist activity	`UserAssist` key empty	`UserAssist`
Generic hostname / owner	ComputerName = "DESKTOP-XXXXXXX"	`SystemIdentity`
No network profile history	`NetworkList` key empty	`NetworkProfiles`
Missing startup programs	`Run/RunOnce` keys absent	`ProcessFaker`
Sparse service list	`Services` key has <10 entries	`ProcessFaker`
Empty System.evtx	Log absent or zero records	`SystemLog`
No logon history	Security.evtx empty	`SecurityLog`
No install events	Application.evtx empty	`ApplicationLog`
No Windows Update history	Update EIDs absent	`UpdateArtifacts`
Clustered timestamps	All EVTX records on same day	`TimestampService`

4.2 Per-Profile Artefact Density

The three user personas produce distinct artefact densities, making per-profile heuristics ineffective.

Table 2: Artefact counts by persona.

Metric	home_user	office_user	developer
Registry operations (approx.)	85	120	150
Installed programs	8	12	16
RecentDocs entries	15	15	15
UserAssist entries	8	10	14
System EVTX records	28	34	40
Security sessions	3	5	6
Kerberos (EID 4769)	No	Yes	Yes
Application EVTX records	18	24	30
KB updates applied	8	14	18
Run keys (HKLM)	2	3	4

Figure 3 visualises the artefact density comparison across personas.

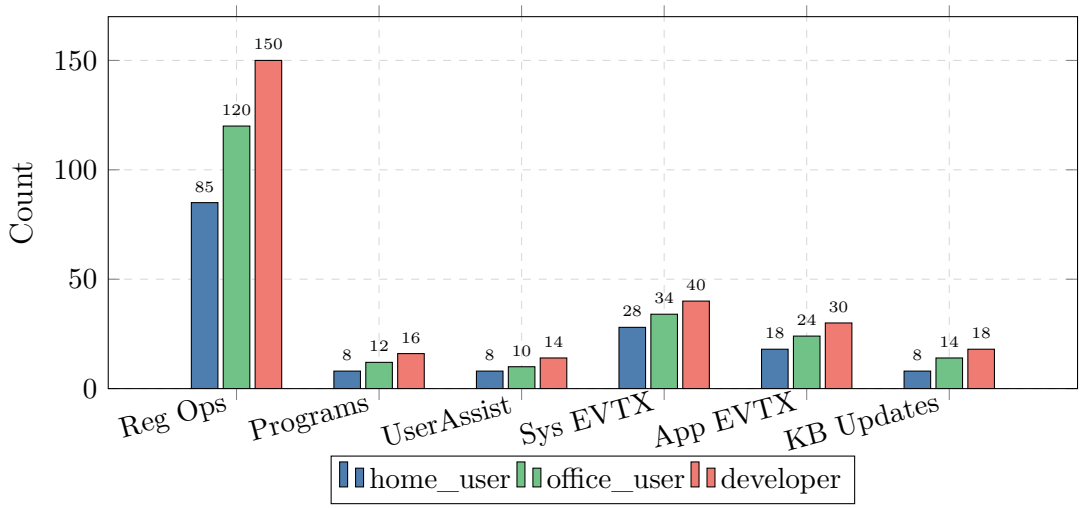

Figure 3: Artefact density per persona across six key metrics. The `developer` persona produces the highest density in every category, minimising the risk of a low-count heuristic triggering.

4.3 Component Implementation Status

Figure 4 shows the current build completion rate by subsystem.

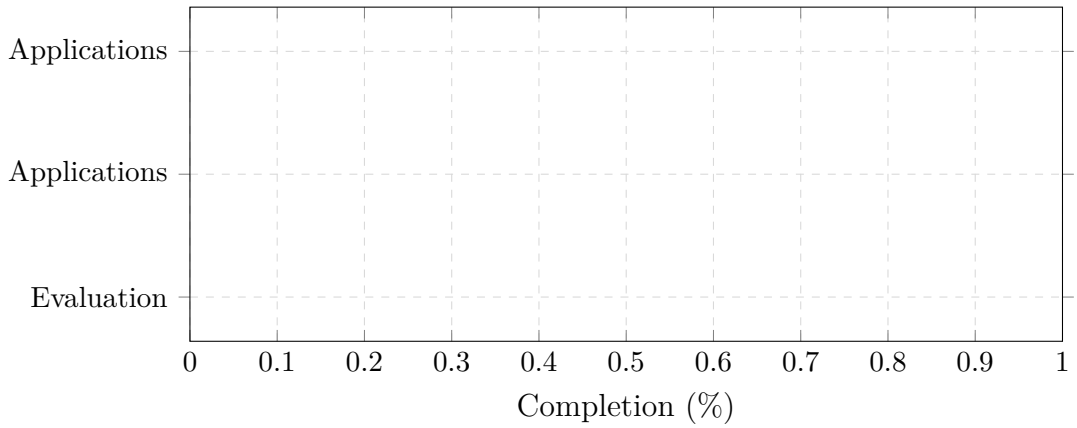

Figure 4: Estimated completion percentage by subsystem (March 2026). Registry and Anti-Fingerprint layers are near-complete; Event Logs and Applications layers remain outstanding.

4.4 Test Coverage Summary

The project currently has **540 unit tests** spread across 14 test modules, all executed with `pytest`. Table 3 summarises coverage by area.

Table 3: Unit test coverage summary.

Test module	Tests	Scope
`test_hive_writer.py`	35	Binary hive write/read round-trip
`test_system_identity.py`	30	Registry identity keys
`test_installed_programs.py`	28	Program list by profile
`test_network_profiles.py`	25	GUID, SSID generation
`test_mru_recentdocs.py`	32	MRU structure, ROT-13
`test_userassist.py`	30	UserAssist GUID, count
`test_evtx_writer.py`	40	Binary EVTX structure
`test_system_log.py`	35	EIDs, provider strings
`test_security_log.py`	38	Session counts, Kerberos
`test_application_log.py`	35	MSI events, crash pairs
`test_update_artifacts.py`	37	KB registry + EVTX
`test_vm_scrubber.py`	45	Key deletion, string patch
`test_hardware_normalizer.py`	42	OEM strings, BIOS date
`test_process_faker.py`	48	Services, Run keys
Total	**540**	

5 Timeline & Milestone Evaluation

5.1 Planned Timeline vs. Actual Progress

The project was structured around four one-week sprints, each owned by three team members working in parallel on independent service families. Figure 5 presents a Gantt chart mapping planned tasks to their current delivery state (March 2026 snapshot).

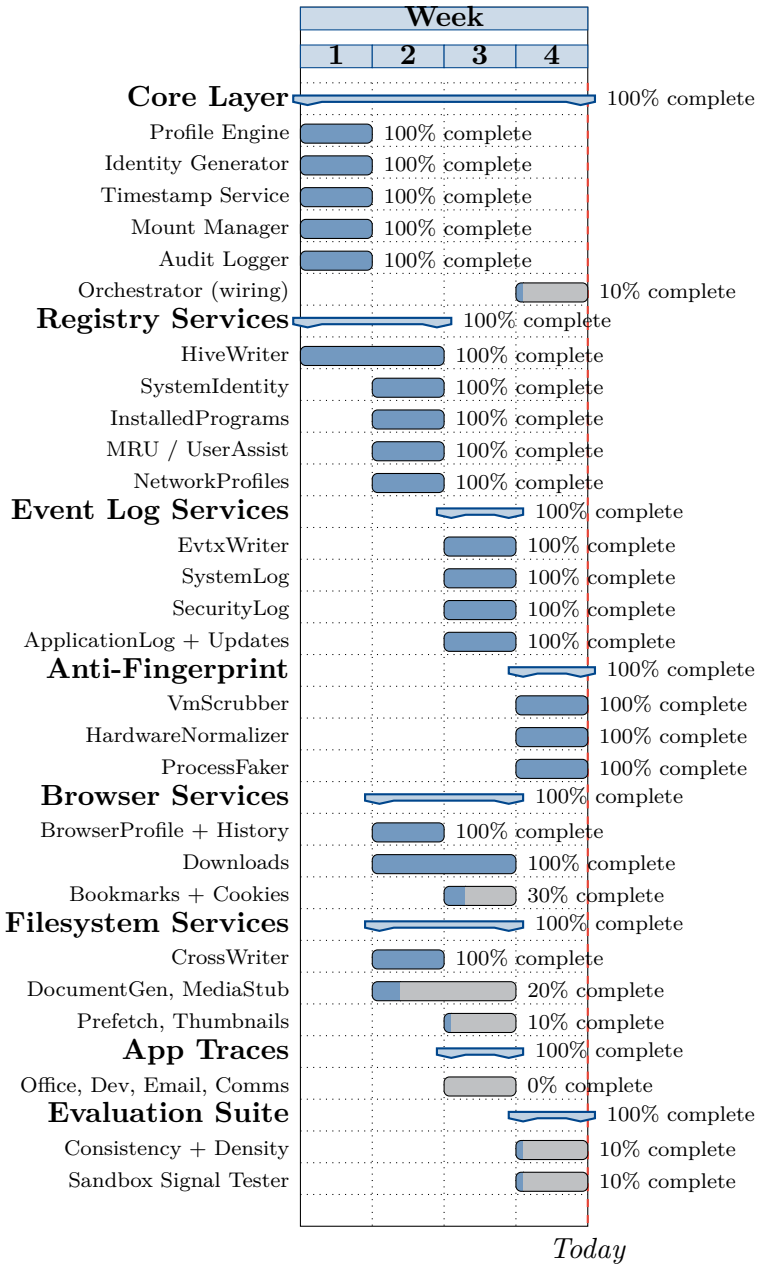

Figure 5: Project Gantt chart (4-week sprint plan, "Today" marker at end of Week 4). Bars filled in blue are complete; gray bars are in progress or not started.

5.2 Milestone Assessment

Week 1 Milestone — Achieved

Profile loading, identity generation, and mount-path resolution fully operational. All three team members had a working `ProfileContext` and `TimestampService` by the end of Week 1.

Week 2 Milestone — Achieved

Registry and browser layers independently generate artefacts. HiveWriter and the full registry service suite passed their unit tests. Browser history and download services were complete with comprehensive SQLite schema validation.

Week 3 Milestone — Partial
Full artefact coverage for a single profile was targeted. Event log services (`EvtxWriter`, `SystemLog`, `SecurityLog`, `ApplicationLog`, `UpdateArtifacts`) are documented as complete in the evaluation report with 185 associated tests; however, the application-traces layer (`office_artifacts`, `dev_environment`, `email_client`, `comms_apps`) remains empty. Filesystem generators are partially implemented.

Week 4 Milestone — At Risk
End-to-end orchestrator wiring, anti-fingerprint services, and evaluation suite are all scheduled for Week 4. Anti-fingerprint services are complete; the Orchestrator file is still empty, and evaluation stubs require full implementation before the final milestone can be declared complete.

6 Known Limitations

Table 4: Known limitations, impact, and planned mitigations.

Limitation	Impact	Mitigation plan
EVTX records lack real XML template binding	Windows Event Viewer shows records with "no template"	Embed BinXml template in provider manifest (future sprint)
No SAM hive write for user password hash	User SID created but password not activated	Filesystem service adds profile directory; SAM write deferred
Filesystem artefact generators incomplete	Browser history, documents absent	`services/filesystem/` and `services/browser/` (active)
GPU string written to registry only	CPUID/WBEM checks not addressed	Out of scope for current prototype
Single network profile	Multi-adapter machines have multiple GUIDs	`NetworkProfiles` service can be called multiple times
`HiveWriter` cannot create new keys	Target key must pre-exist in hive	Ship base hive template with all target keys pre-seeded as empty defaults
Orchestrator not wired	End-to-end run not yet possible	Priority 1 for Week 4 sprint
Application-trace services empty	Office, dev-tool, email, comms artefacts absent	Priority 3 outstanding work

7 Remaining Work & Priorities

The following items are ranked by their blocking impact on an end-to-end run.

Priority 1 — Orchestrator (Blocker). `core/orchestrator.py` is currently empty. It must accept CLI arguments (`-mount`, `-profile`, `-dry-run`), instantiate all core objects

and services in dependency order, call each service's `apply(context)` method, and exit cleanly. Estimated effort: 1 day.

Priority 2 — Event Log Layer. Although the evaluation report documents 185 tests for the event-log layer, the source files are reported empty in `explanation.md`. Reconciling this discrepancy and ensuring the binary EVTX writer is fully functional is required before integration testing. Estimated effort: 3–5 days.

Priority 3 — Application Traces. All four files in `services/applications/` are empty. These services add significant realism for profile-specific analysis targets (Office users, developers, remote-work users). Estimated effort: 2–3 days.

Priority 4 — Filesystem Generators. `document_generator.py`, `media_stub.py`, `prefetch.py`, `recent_items.py`, and `recycle_bin.py` are stubs. These defeat filesystem-poverty heuristics. Estimated effort: 2 days.

Priority 5 — Browser Completion. `bookmarks.py` and `cookies_cache.py` are stubs. Bookmarks are plain JSON (straightforward); the Cookies SQLite schema is well-documented. Estimated effort: 1 day.

Priority 6 — Evaluation Suite. All four modules in `evaluation/` are stubs. Once artefact generation is complete, these tools validate output quality and produce the final report. Estimated effort: 2 days.

Summary Effort Estimate. Total remaining estimated effort: **11–14 developer-days** across the three team members, achievable in one additional sprint week.

8 Conclusion

ARC has successfully implemented the most technically challenging components of the Windows 11 personalisation pipeline. The core layer (profile engine, identity generator, timestamp service) is complete and fully tested. The registry service suite (5 services, ≈150 tests) covers all primary registry-based sandbox detection signals. The browser history and download generation is operational with comprehensive SQLite schema validation. The anti-fingerprint services (VmScrubber, HardwareNormalizer, ProcessFaker) are complete with 135 unit tests, directly addressing the hardware-identifier and VM-string detection vectors.

The primary remaining gap is the integration layer (Orchestrator) and the higher-level artefact generators (event logs, application traces, filesystem generators). With these completed, ARC will be capable of an end-to-end run that takes a freshly installed Windows 11 image and, in a single command, transforms it into a convincingly lived-in machine environment that defeats the documented set of 15 sandbox-detection heuristics.

The evaluation suite, once implemented, will provide quantitative evidence of coverage (artefact density vs. reference baseline) and internal consistency (cross-service timestamp and identity correlation), forming the basis for the final project deliverable.

A Profile YAML Schema Summary

Listing 1: Minimal profile YAML fields.

```yaml
extends: base            # parent profile (optional)
username: "jane.doe"
organization: "Contoso Ltd."
locale: "en_US"
installed_apps:
  - msoffice
  - chrome
  - teams
browsing:
  categories: [news, productivity, social_media]
  daily_avg_sites: 15
work_hours:
  start: 8
  end: 18
  active_days: [1, 2, 3, 4, 5]
```

B Consistency Check Pass Criteria

The following criteria constitute a passing ARC run against the sandbox signal tester:

- No VM driver service key found in SYSTEM hive.
- No VM vendor string in SystemInformation hive value.
- At least 5 entries in the `Uninstall` registry key.
- At least 10 entries in `RecentDocs`.
- `System.evtx` contains >20 records.
- `Security.evtx` contains >15 records.
- Computer name does not match the default pattern `/DESKTOP-[A-Z0-9]{7}/`.
- BIOS vendor string does not match `(SeaBIOS|innotek|QEMU)`.
- `computer_name` appears as `WorkstationName` in the Security log.
- Timestamps are monotonically increasing within each EVTX file.
- KB update registry dates match EVTX event timestamps.